Arora: a Portable Runtime for Interoperable Robotics Applications

Victor Paléologue, Andrew Schoen, Saad Elbeleidy, Ross Mead

Semio

{victor.paleologue, andy.schoen, saad.elbeleidy, ross.mead}@semio.ai

Abstract

Robot software is heterogeneous: functionality arrives as components that must each be integrated, the middleware beneath them is plural, and an integrator working across robots writes an adaptation layer once per platform. Robotics developers and users ask for a higher level of abstraction, for software decoupled from hardware, and for reusable components behind standardized interfaces. We present Arora, an open-source device runtime that answers those needs in one place. An Arora device is one process around one data store, which hardware, external clients, behavior and computation reach through four interfaces. Capabilities, including control policies and animation, are WebAssembly modules built once, and running anywhere, including in a browser. This should ease the writing and testing of behaviors away from the physical robot and its middleware. Behaviors are written as either behavior trees or dataflow graphs, on top of lower-level interfaces that are agnostic from other middleware. We close on Vizij, an earlier attempt to standardize interactive behaviors through face rigging, and on the points where Arora converges with it.

Introduction

The robot software landscape is heterogeneous, to the point it incurs real costs for companies who engineer robotics solutions. In a study of the service robotics industry (García et al. 2020), interoperability was named a challenge by 61% of respondents, the lack of standardized solutions by 57%, and software reuse by 52%. Functionality frequently exists as components that must each be integrated. A multiple-case study in the same domain (García et al. 2022) attributes this to variability of customer focus, hardware, and operating system environments, and as a result, *ad hoc* practices prevail, challenging reuse and maintenance. Moreover, the middleware beneath those components of functionality is itself heterogeneous (Mohamed, Al-Jaroodi, and Jawhar 2008; Elkady and Sobh 2012; Tsardoulias and Mitkas 2017), and interoperating two middlewares remains bespoke work (Aragão, Moreno, and Bernardino 2016). An integrator working across robots writes that adaptation layer once per platform. Their recommendation is to raise the level of abstraction, to decouple software from hardware, and to organize reusable components behind harmonized and standardized interfaces.

In turn, the recommendation leads to writing, running and testing behaviors away from the robot and away from its middleware, for instance in a web application, or on a desktop app. They are then deployed unchanged into the middleware the robot already runs. This is not a convenience: validating robotic systems is the second most reported challenge in that same industry study, ahead of the simulation-to-real transition (García et al. 2020).

In terms of behavior specification, integrators frequently request behavior trees, because they both sequence and terminate, resulting in their growing use in open-source robotics (Ghzouli et al. 2023). They appear useful alongside other types of finite-state machines, where there are many sources of variability (García et al. 2022). But at lower levels of abstraction, dataflow graphs can be a better and established way to structure pieces of robot applications in order to coordinate precise motions and behaviors (Zhang et al. 2026). We draw a parallel between this and the rigging in animation: a rig is a function from a few control values to many values (Kalegina et al. 2018), and a dataflow graph is suited to represent that transformation. This matters for animators, who carry much of the work of making robot behaviors expressive and acceptable.

Arora is our attempt to answer these needs, in one open-source device runtime written in Rust.[1] In this paper we will review the landscape around robotics software standardization, then detail how Arora distinguishes itself and what its core concepts are. We then report how we made it converge with Vizij, an earlier attempt at standardizing interactive behaviors through face rigging.

Related Work

Robot software frameworks have argued for the same separations for two decades: Player exposed devices through an abstraction server (Gerkey, Vaughan, and Howard 2003), and YARP argued for code that outlives both its hardware and its transport (Fitzpatrick, Metta, and Natale 2008). ROS 2 makes the unit of composition a process and the unit of communication a typed topic (Macenski et al. 2022),

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

[1] Arora is open source under the MIT license: https://github.com/semio-ai/arora-sdk

and Zenoh unifies communication across that range (Corsaro et al. 2023). Hardware abstraction is usually per-actuator (Chitta et al. 2017), though R-HAL abstracts across robots through a software framework rather than per-robot drivers (Rigano et al. 2018).

Several systems have arisen to address these goals. DORA (Zhang et al. 2026) proposes a dataflow-oriented architecture for low-overhead composition. HARMONI (Spitale et al. 2021) composes perception, dialogue and behavior modules for interaction, at one node per module. A measurement of asynchronous Rust in ROS 2 applications (Škoudlil, Sojka, and Hanzálek 2025) is the nearest published evidence that a Rust robot runtime is practical. Cyber-physical WebAssembly (Van Kenhove et al. 2025) arrives at the same arrangement independently, with pluggable Wasm drivers behind hardware interfaces.

Copper (The Copper Project 2026) checks several of these needs altogether: a Rust runtime that schedules its tasks in one process, records a run for deterministic replay, and the ability to recompile the same application for the browser. It proposes task graphs, similar to ROS graphs, wired at compile time hooking function calls to published events. An entire ROS 2 stack has also been compiled to WebAssembly (Fischer et al. 2025), which brings the middleware to the browser rather than the application.

Behavior formalisms are standardized and then compared as alternatives (Colledanchise and Ögren 2018; Bohren and Cousins 2010; Ghzouli et al. 2023), and authoring tools are usually bound to the formalism they emit (Porfirio et al. 2023). Motion systems, including our own, own the motion layer (Schoen et al. 2023), where Arora leaves the policies and the rule that combines them to modules.

Toolkits for social robots supply a runtime together with the representation it accepts (Ribeiro et al. 2016; Alves-Oliveira et al. 2022; Dennler et al. 2024), which is what makes them hard to compose with each other. Finally, our earlier work on Vizij (Elbeleidy et al. 2025) applies dataflow graphs to the rigging of faces and standardizes the interfaces at several layers of face design. Vizij is a runtime that interprets such graphs to produce complex behaviors on faces. It comes as a stand-alone app that can display the face directly, or embedded into web apps, such as the face authoring app itself. The latest developments have operated a convergence of Vizij with Arora, making Vizij itself a good illustration of Arora. We mention it regularly throughout this paper, and return to that in the last section.

In the next section we detail what Arora is, and how it distinguishes itself from the systems reviewed here.

Arora

Arora is a robotics framework designed with the vision that the components, called modules, of a deployed system may be loaded, unloaded or swapped at runtime. It pushes further the commitment to portability by proposing that any functionality, not only hardware-side drivers, may be added through WebAssembly modules, exposing only functions. Each recommendation of the studies lands on one part: the authored graph raises the level of abstraction, the Hardware Abstraction Layer (HAL) decouples software from hardware, and modules behind the interfaces are the reusable components.

The runtime is a main loop that polls the components via a behavior interpreter, parsing a graph that can be swapped at runtime, and which is not restricted to either behavior trees or dataflow graphs. Figure 1 shows one device, its components and the order in which the loop reaches them. Hence the application running with Arora can bring its custom functionalities as modules, and control their execution precisely via the behavior graph. The graph connects the data keys to the inputs and outputs of the module functions.

Deploying the application to new devices does not require recompilation. Arora is compiled once and for all with the proper hardware abstraction layer (HAL), to be then used for a variety of applications. It usually runs as a single process on a device, but can also be run several times within a single process, for instance for applications simulating several devices at a time.

In short, Arora resembles Copper with more dynamic components and graph execution, based on known solutions such as WebAssembly, behavior trees and dataflow graphs. Arora's pre-built components are:

- the data store;
- the behavior interpreter;
- the engine hosting the modules;
- the HAL, for driving the devices;
- the bridges, for interoperability and for accepting remote clients.

We detail these components in the next subsections.

One Store, Four Interfaces

A device is one instance of the looping runtime. Each step of that loop drains what arrived since the previous one, then ticks the behavior interpreter, collapsing the frame's writes into one state change before writing that change out, by default every 10 ms. Every transfer in the loop is non-blocking, and the data store, which addresses every value by a key, has a single owner, so the data plane holds no lock.

Four interfaces converge on the data store (Figure 1). They formalize the roles of the core components of software for robotics, and as such are a proposition for standardization. The HAL is the hardware boundary. A bridge is a remote endpoint that takes incoming data and information that while sending its own. The behavior interpreter is ticked once per step, and the engine loads and calls modules. No interface names another directly, and instead communication is handled at the data store. That constraint is what makes each interface replaceable: a bridge is added or removed without touching behavior, and a behavior is written without knowing which transport carries its outputs. The four areas this paper is named for sit on those interfaces: control policies and animation are modules behind the engine, control the robot already owns stays behind the HAL, behavior is the interpreter, and inspection and teleoperation operate through the use of bridges.

A capability is reached by a function call on a key, not by a message on a middleware topic. One device is one process:

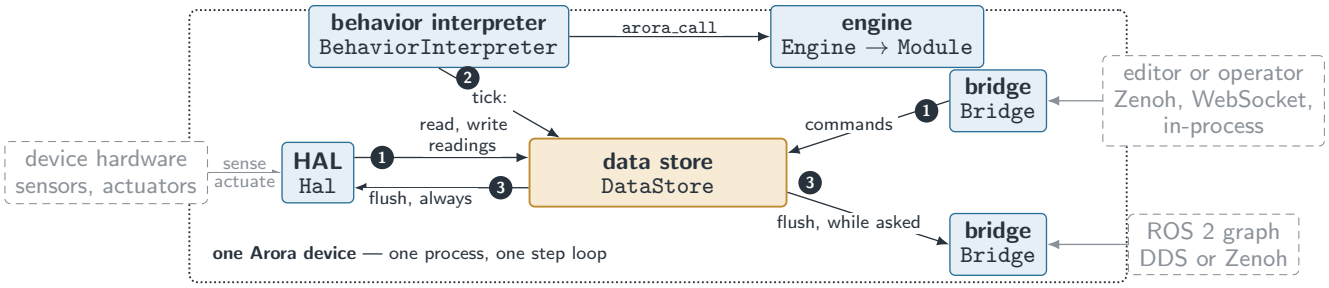


Figure 1: An Arora device. Boxes are components and interfaces, named as in the code, and the numbers mark the three phases of one step: (1) drain hardware readings and bridge commands into the store, (2) tick the behavior interpreter, (3) flush the frame's writes out. The interfaces meet only in the store.

a faulting module is not isolated by the operating system, but the tick order is defined and no inter-process transport sits on the data plane.

Modules: One Binary Format, Two Hosts

A module is the lower-level unit of reuse — the component of the practitioner studies, packaged so that it can be integrated without being ported. A module declares its functions in a manifest, from which bindings are generated for Rust and for C++, and builds to either of the two current WASI targets. The same artifact is instantiated either by a native WebAssembly runtime, or by the browser's own implementation with the few WASI calls stubbed. The device itself also compiles to WebAssembly and is exposed to JavaScript, so a page hosts a device instead of talking to one. We use WebAssembly as an interoperable binary format (Haas et al. 2017), not as a sandbox for untrusted code: the guest is the author's own, and what it buys is one build.

The runtime is written in Rust, well-known for its safety. Unsafe code is confined to the buffer codec that crosses the guest boundary and to the engine, where Rust's guarantees stop (Jung et al. 2021).

Control and Animation

Kinematics, a trajectory generator, an inverse-kinematics step, a gaze solver or a learned policy exist as functions from one set of values to another set of values. Arora provides each as a module, called by identifier from the behavior graph, which gives it a typed call signature and a version. This makes a policy composable rather than simply present.

This first publication about Arora is marked by the porting of Vizij to Arora, and specifically of its deterministic animation engine, now packaged as an Arora module. It is then invoked by the behavior graph as a function, and ported to work in Vizij's web applications. Clips and authored programs travel inside the Vizij graphs and modules, while Arora hosts them with no custom code to support them.

Some control is already behind the HAL. Arora attaches to robots that arrive with their own controllers, wrapped by their own robotics software. Hence Arora does not intend to take full control of the hardware, but instead accommodates how the hardware features are presented, e.g. how the ROS 2 graph is organized, or what HTTP REST API may be available.

This is a different decomposition from a controller manager, which places control above the driver in the same process (Chitta et al. 2017). In Arora it sits on either side of that boundary, and which side is a property of the robot rather than of the runtime.

Behavior

Behaviors are defined as generic node-based graphs. No interpretation or traversal rule is inherently attached. This unifies the communication of changes in the graph from external editors.

It is then the behavior interpreters that give meaning to the graph. For instance the behavior tree interpreter reads links as child and argument edges, and returns a status that can terminate. Vizij's node-graph interpreter reads the same links as dataflow, and stays running. Both implement the behavior interface, but may have specific functions. For instance the behavior trees can be imported from the XML dialect of the established behavior-tree tooling (Ghzouli et al. 2023).

Editing is not a restart: the behavior interpreter module accumulates the graph diffs it receives, and they are applied to the internal model on the next tick. Loading a behavior is the same operation applied to an empty graph. Here the formalism is a property of the device.

A behavior is where data keys are put in relation with functions. Contrary to more common practice in the ROS community, Arora discourages the hard-coding of keys (or topic names) as input or output of functions. This favors reusability and flexibility, and allows concurrent calls of the same function to output to different keys.

Behavior is also where policies would be combined. Blending two policies, switching between them and deciding which one wins are not properties of a policy. They are decisions of the application's behavior. Vizij's node graph already carries operators for that: blend nodes with weighted averages over weights and masks, transition nodes that damp and slew a value toward a target.

The vocabulary is not fixed by the runtime either. A module can provide node types, including composites that receive their children and tick them back through the engine, so a module controls when and whether each child runs. A

Transport	Representation	In SDK
WebSocket	JSON keys and values	yes
ROS 2 (DDS or Zenoh)	keys to topics, methods to services, task runs to actions	yes
Studio protocol	device-client protocol	no †
In-process	direct calls, no serialization	yes

Table 1: Bridge implementations. Each is one bridge, and a device holds several at once. †An external crate behind a feature flag, not code in this repository.

new arbitration rule — a priority mux, a subsumption layer — is therefore a module, authored and versioned like any other, and not a change to the runtime. We state this as intent: what exists today is the node vocabulary in Vizij and the module-composite mechanism in Arora, demonstrated on faces.

Inspection and Teleoperation

Inspecting a device and teleoperating it are the same operation at the same interface, and neither is an application. A remote asks a device for its keys and method signatures, and receives them as data.

Teleoperation tooling is usually an application against a robot's API (Elbeleidy, Mott, and Williams 2022), which ties it to one robot.

Semio Studio was built around Arora and a specific bridge.[2] Because the same core code can run in a browser and on a device, behaviors are envisioned to be authored and tested directly in the web app, or deployed to an actual device running Arora. Studio also hosts a growing collection of shared assets — models, faces, behaviors — and, in the future, modules that can be selected and integrated.

Table 1 lists the transports. The runtime depends on the bridge interface rather than on any one of them, and that interface was modelled on the device-client of Semio Studio's protocol (Mead 2017), so that protocol could be one implementation among others rather than the interface itself.

[2]Semio Studio is free to use at: https://studio.semio.ai/



Figure 2: Two face assets rendered by the same device, from the repository's reference renders. Each asset carries its own rig and its own authored graphs, and both accept the same control vocabulary, so a behavior written for one drives the other. The left rig is a face for an open social robot platform (Specian et al. 2022).

Convergence with Vizij

Arora grew from robotics-centered ideas and had remained unpublished. Vizij took from it the idea that the graphs tie keys together, and that functionality should not be permanently attached to them: behavior (node graph) has full control of inputs and outputs attached to functions. In the latest iteration of Arora we complied with Vizij's interfaces, such as the data storage, bridge design, and its face rendering.

As a result their abstractions converged, so that Arora could serve a face and a robot at once. Vizij adopted Arora's value type through a compatibility layer, Arora's data store interface. The Vizij keys are properly namespaced to leave room for more robotics-centered behaviors.

Keyframe animation was built into Vizij and is an Arora module. Where Vizij carries a concept the shared types have no field for (units, ranges and color spaces) it survives as a reserved sidecar namespace in the same store.

Finally, by adopting the same bridge interfaces, Vizij gained connectivity to ROS 2 and Semio Studio.

Discussion

Arora is one process around a data store, with a behavior interpreter, an engine hosting the modules, a HAL and bridges as its only pre-conceived components. Everything else is a module the application brings, connected by a graph authored rather than compiled in. It answers the needs the studies report: a higher level of abstraction, software decoupled from hardware and reusable components behind standardized interfaces, without fixing the middleware, the behavior notation or the deployment target.

What the artifact does not yet support bounds that claim. The repository ships one behavior interpreter, and the node-graph interpreter that makes the notation argument concrete lives in Vizij. No published case arbitrates several whole-body control policies on a robot, and the only arbitration the runtime defines is a priority field on the requests the ROS 2 bridge accepts, which resolves requests rather than policies. Introspection over the ROS 2 bridge is in progress, the WebSocket bridge is unauthenticated and loopback-only by default, and distributing modules and shared assets through Studio is ongoing work.

As part of this discussion, there are some open questions. Where should a shared authored graph stop: are nodes, ports and links with no traversal rule enough, or does usefulness require a semantics? What is the right unit of interoperability for a capability, a message type, a process, or a binary module with a typed call signature? Which well-known keys and module functions should we support next, to pursue further standardization and enable the reuse of behavior graphs?

Disclosure of AI Assistance

An AI assistant drafted parts of this text and edited throughout, under the authors' direction. It also surveyed the repositories described here, checked the bibliography against DOI and venue records, and prepared the figures. The authors determined the architecture, the claims and the structure, verified every statement against the source code, and are responsible for the content. No AI system is an author of this paper.

References

Alves-Oliveira, P.; Mihata, K.; Karim, R.; Bjorling, E. A.; and Cakmak, M. 2022. FLEX-SDK: An Open-Source Software Development Kit for Creating Social Robots. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology*, 1–10. ACM.

Aragão, M.; Moreno, P.; and Bernardino, A. 2016. Middleware Interoperability for Robotics: A ROS–YARP Framework. *Frontiers in Robotics and AI*, 3.

Bohren, J.; and Cousins, S. 2010. The SMACH High-Level Executive [ROS News]. *IEEE Robotics & Automation Magazine*, 17(4): 18–20.

Chitta, S.; Marder-Eppstein, E.; Meeussen, W.; Pradeep, V.; Rodríguez Tsouroukdissian, A.; Bohren, J.; Coleman, D.; Magyar, B.; Raiola, G.; Lüdtke, M.; and Fernandez Perdomo, E. 2017. ros_control: A generic and simple control framework for ROS. *The Journal of Open Source Software*, 2(20): 456.

Colledanchise, M.; and Ögren, P. 2018. *Behavior Trees in Robotics and AI: An Introduction*. Chapman & Hall/CRC Artificial Intelligence and Robotics Series. CRC Press. Preprint: arXiv:1709.00084.

Corsaro, A.; Cominardi, L.; Hecart, O.; Baldoni, G.; Avital, J. E. P.; Loudet, J.; Guimares, C.; Ilyin, M.; and Bannov, D. 2023. Zenoh: Unifying Communication, Storage and Computation from the Cloud to the Microcontroller. In *2023 26th Euromicro Conference on Digital System Design (DSD)*, 422–428. IEEE.

Dennler, N. S.; Torrence, E.; Yoo, U.; Nikolaidis, S.; and Mataric, M. 2024. PyLips: an Open-Source Python Package to Expand Participation in Embodied Interaction. In *The 37th Annual ACM Symposium on User Interface Software and Technology*, 1–4. ACM.

Elbeleidy, S.; Mott, T.; and Williams, T. 2022. Practical, Ethical, and Overlooked: Teleoperated Socially Assistive Robots in the Quest for Autonomy. In *2022 17th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, 577–587. IEEE.

Elbeleidy, S.; Schoen, A.; Birmingham, C.; Ribeiro, T.; Paléologue, V.; Dooley, D.; and Mead, R. 2025. Vizij: Supporting the design, animation, and deployment of rendered robot faces. In *Proceedings of the 3rd AAAI Fall Symposium on Unifying Representations for Robot Application Development (UR-RAD)*. Arlington, VA, USA. https://ur-rad.github.io/papers/vizij-design-animation-deployment-faces/.

Elkady, A.; and Sobh, T. 2012. Robotics Middleware: A Comprehensive Literature Survey and Attribute-Based Bibliography. *Journal of Robotics*, 2012: 1–15.

Fischer, T.; Paredes, I.; Batchelor, M.; Beier, T.; Haviland, J.; Traversaro, S.; Vollprecht, W.; Schmitz, M.; and Milford, M. 2025. ROS2WASM: Bringing the Robot Operating System to the Web. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 15167–15173. IEEE.

Fitzpatrick, P.; Metta, G.; and Natale, L. 2008. Towards long-lived robot genes. *Robotics and Autonomous Systems*, 56(1): 29–45.

García, S.; Strüber, D.; Brugali, D.; Berger, T.; and Pelliccione, P. 2020. Robotics software engineering: a perspective from the service robotics domain. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 593–604. ACM.

García, S.; Strüber, D.; Brugali, D.; Fava, A. D.; Pelliccione, P.; and Berger, T. 2022. Software variability in service robotics. *Empirical Software Engineering*, 28(1): 24.

Gerkey, B.; Vaughan, R. T.; and Howard, A. 2003. The Player/Stage Project: Tools for Multi-Robot and Distributed Sensor Systems. In *Proceedings of the 11th International Conference on Advanced Robotics (ICAR)*, 317–323.

Ghzouli, R.; Berger, T.; Johnsen, E. B.; Wasowski, A.; and Dragule, S. 2023. Behavior Trees and State Machines in Robotics Applications. *IEEE Transactions on Software Engineering*, 49(9): 4243–4267.

Haas, A.; Rossberg, A.; Schuff, D. L.; Titzer, B. L.; Holman, M.; Gohman, D.; Wagner, L.; Zakai, A.; and Bastien, J. 2017. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 185–200. ACM.

Jung, R.; Jourdan, J.-H.; Krebbers, R.; and Dreyer, D. 2021. Safe systems programming in Rust. *Communications of the ACM*, 64(4): 144–152.

Kalegina, A.; Schroeder, G.; Allchin, A.; Berlin, K.; and Cakmak, M. 2018. Characterizing the Design Space of Rendered Robot Faces. In *Proceedings of the 2018 ACM/IEEE International Conference on Human-Robot Interaction*, 96–104. ACM.

Macenski, S.; Foote, T.; Gerkey, B.; Lalancette, C.; and Woodall, W. 2022. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66).

Mead, R. 2017. Semio: Developing a cloud-based platform for multimodal conversational AI in social robotics. In *2017 IEEE International Conference on Consumer Electronics (ICCE)*, 291–292. IEEE.

Mohamed, N.; Al-Jaroodi, J.; and Jawhar, I. 2008. Middleware for Robotics: A Survey. In *2008 IEEE Conference on Robotics, Automation and Mechatronics (RAM)*, 736–742. IEEE.

Porfirio, D.; Stegner, L.; Cakmak, M.; Sauppé, A.; Albarghouthi, A.; and Mutlu, B. 2023. Sketching Robot Programs On the Fly. In *Proceedings of the 2023 ACM/IEEE International Conference on Human-Robot Interaction*, 584–593. ACM.

Ribeiro, T.; Paiva, A.; Tome, D.; and Pereira, A. 2016. The SERA Ecosystem: Socially Expressive Robotics Architecture for Autonomous Human-Robot Interaction. In *AAAI Spring Symposium Series*. DBLP conf/aaaiss/0001PTP16.

Rigano, G.; Muratore, L.; Laurenzi, A.; Hoffman, E. M.; and Tsagarakis, N. 2018. Towards a Robot Hardware

Abstraction Layer (R-Hal) Leveraging the XBot Software Framework. In *2018 Second IEEE International Conference on Robotic Computing (IRC)*, 175–176. IEEE.

Schoen, A.; Sullivan, D.; Zhang, Z. D.; Rakita, D.; and Mutlu, B. 2023. Lively: Enabling Multimodal, Lifelike, and Extensible Real-time Robot Motion. In *Proceedings of the 2023 ACM/IEEE International Conference on Human-Robot Interaction*, 594–602. ACM.

Specian, A.; Mead, R.; Kim, S.; Mataric, M.; and Yim, M. 2022. Quori: A Community-Informed Design of a Socially Interactive Humanoid Robot. *IEEE Transactions on Robotics*, 38(3): 1755–1772.

Spitale, M.; Birmingham, C.; Swan, R. M.; and Mataric, M. J. 2021. Composing HARMONI: An Open-source Tool for Human and Robot Modular OpeN Interaction. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 3322–3329. IEEE.

The Copper Project. 2026. Copper: an operating system for robots. Software, Apache-2.0, https://github.com/copper-project/copper-rs. No accompanying publication as of this writing.

Tsardoulias, E.; and Mitkas, P. A. 2017. Robotic frameworks, architectures and middleware comparison. arXiv:1711.06842 [cs.RO].

Van Kenhove, M.; Seidler, M.; Vandenberghe, F.; Dujardin, W.; Hennen, W.; Vogel, A.; Sebrechts, M.; Goethals, T.; De Turck, F.; and Volckaert, B. 2025. Cyber-Physical WebAssembly: Secure Hardware Interfaces and Pluggable Drivers. In *NOMS 2025-2025 IEEE Network Operations and Management Symposium*, 1–7. IEEE.

Škoudlil, M.; Sojka, M.; and Hanzálek, Z. 2025. A first look at ROS 2 applications written in asynchronous Rust. In *37th Euromicro Conference on Real-Time Systems (ECRTS 2025)*, LIPIcs. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Preprint: arXiv:2505.21323.

Zhang, X.; Lv, B.; Tao, X.; Wang, X.; Bao, J.; He, Y.; Chen, Y.; and Yang, Z. 2026. DORA: Dataflow Oriented Robotic Architecture. arXiv:2602.13252 [cs.RO].